

Household Energy Consumption Prediction

Project Walkthrough — Notebook Explained Line by Line, For Team Members New to ML

This document explains, in plain language, what our machine learning notebook does, why we made each decision, which experiments worked, which failed (and why that's still useful), and what our final result is. No prior ML knowledge is assumed.

1. What Are We Actually Trying To Do?

Our project predicts how much electricity a household's appliances will use (measured in Watt-hours, Wh) based on things we can measure: room temperatures, humidity levels, outdoor weather, and the time of day. This is called a **regression problem** — we're not sorting things into categories (like spam / not spam), we're predicting a number.

Why does this matter? If a model can predict energy usage accurately from sensor readings, it can help detect abnormal consumption, plan for peak demand, and eventually be built into a smart-home app that gives people real-time energy insights.

2. The Dataset

We use a well-known public dataset (originally collected by Candanedo et al., and reused in our base research paper). It contains 10-minute-interval sensor readings from a low-energy house over about 4.5 months — roughly 19,735 rows. Each row has:

- **Appliances** — our target: energy used by appliances (Wh). This is what we're predicting.
- **T1–T9, RH_1–RH_9** — temperature and humidity in 9 different rooms of the house.
- **T_out, RH_out, Press_mm_hg, Windspeed, Visibility, Tdewpoint** — outdoor weather station readings.
- **lights** — energy used by light fixtures (dropped later — explained below).
- **date** — timestamp of the reading, used to build time-based features.
- **rv1, rv2** — two random numbers the dataset creators added on purpose, to test whether models pick up noise. We always dropped these.

3. ML Concepts Explained Simply

Before the walkthrough, here are the core terms you'll see everywhere in the notebook and in this document.

Training data vs. Testing data

We never let the model see all the data during training. We split it — usually 70-80% to **train** (teach the model patterns) and the rest to **test** (check if it actually learned something useful, on data it has never seen). This mimics real-world use: the model will one day face new data it wasn't trained on.

R^2 Score (R-squared / coefficient of determination)

This is our main scorecard. It answers: 'What fraction of the ups-and-downs in energy usage does our model actually explain?' It ranges from 0 to 1 (sometimes negative if the model is worse than just guessing the average every time).

$R^2 = 1.0$

Perfect prediction — extremely rare and often a red flag for a data leak.

$R^2 = 0.70$	Model explains 70% of the variation in energy usage. Good for noisy real-world behavioral data.
$R^2 = 0.0$	Model is no better than always guessing the average value.
$R^2 < 0$	Model is actively worse than just guessing the average — something is wrong.

RMSE (Root Mean Squared Error)

This tells us, in the same units as our target (Wh), roughly how far off our predictions are on average — but it punishes big mistakes more heavily than small ones (because it squares the errors before averaging). Lower RMSE = better. An RMSE of 58 means our typical prediction is off by around 58 Wh.

MAE (Mean Absolute Error)

Similar to RMSE but simpler: the plain average of how far off each prediction was, without squaring. It's easier to explain to non-technical people: 'On average, we're off by X Wh.'

MAPE (Mean Absolute Percentage Error)

Same idea as MAE, but expressed as a percentage of the actual value. Useful for comparing error sizes across very different scales.

Overfitting

When a model memorizes the training data instead of learning general patterns. It looks excellent on training data but performs badly on new/test data. We watch for this by always checking performance on the held-out test set, not the training set.

Data Leakage

A serious and easy-to-miss mistake where information from the 'future' or from the answer itself accidentally sneaks into the model's input features. It creates unrealistically high scores that fall apart in real use. We ran into this ourselves — see Section 5.

Feature Engineering

Creating new, more useful input columns from the raw data — e.g., turning a timestamp into 'hour of day', or creating a column that stores 'appliance usage 10 minutes ago' so the model can use recent history to help predict the next reading.

Ensemble / Stacking

Instead of trusting one model, we train several different models and combine their predictions (by averaging, or by training a small extra model that learns the best way to combine them). This usually — but not always — improves accuracy.

4. Step-by-Step Walkthrough of the Notebook

Step 1 — Load the data

We load the JSON file into a table (Pandas DataFrame) and check its shape, column names, data types, and confirm there are no missing values or duplicate rows. This is basic 'hygiene' before any modelling — garbage in, garbage out.

Step 2 — Extract time features

The 'date' column by itself is not directly usable by most ML models. We pulled out Hour, Day, Month, Weekday, and a Weekend flag (1 if Saturday/Sunday). Why? Because energy usage has strong daily and weekly patterns (e.g. more usage in the evening, different patterns on weekends).

Step 3 — Cyclical encoding of time

A raw 'Hour' column has a hidden bug: to a model, hour 23 and hour 0 look far apart numerically, even though midnight is actually right next to 11pm on a clock. We fixed this by converting Hour, Weekday, and Month into sine/cosine pairs — a standard trick that correctly represents 'wrap-around' time.

Step 4 — Lag and rolling features

This was the single biggest driver of our early improvement. We added columns like 'Appliances_lag_1' (what was appliance usage 10 minutes ago?) and 'Appliances_roll_3' (average usage over the last 3 readings). Energy usage right now is strongly related to energy usage a few minutes ago — this feature captures that.

Step 5 — Train/test split and baseline models

We split data 80/20 and trained four baseline models with default settings: Linear Regression, Random Forest, Extra Trees, and XGBoost. Baseline R^2 scores ranged 0.61–0.65 — establishing a benchmark before any tuning.

Step 6 — Hyperparameter tuning (RandomizedSearchCV)

Each model has 'settings' (hyperparameters) like how many trees to build, how deep each tree can go, etc. Rather than guess, RandomizedSearchCV automatically tries many combinations and keeps the best one, checked via cross-validation (splitting training data into folds and testing on each). This nudged our best XGBoost model to $R^2=0.656$.

Step 7 — Feature importance check

We asked the trained model which input columns it relied on most. This is a sanity check — if the model leaned entirely on something suspicious, that's a red flag. Here it confirmed temperature, humidity, and our lag/rolling features were doing the real work.

5. Our Journey to Improve R^2 (Including Dead Ends)

A key part of doing ML honestly is showing what we tried and why some things didn't work. This is exactly the kind of process a reviewer wants to see — it shows real understanding, not just chasing a high number.

Attempt 1 — Ensembling (averaging 3 models)

We trained XGBoost, HistGradientBoosting, and Random Forest, then averaged their predictions. Result: R^2 actually dropped slightly (0.631). **Why it didn't help:** all three models were seeing the exact same features and making very similar mistakes — averaging near-identical opinions doesn't add new information.

Attempt 2 — Outlier capping + performance-weighted ensemble

We capped the top 1% of extreme appliance-usage spikes and weighted each model's vote by its own accuracy. Result: $R^2=0.663$ — a small improvement, but still close to our original baseline. This confirmed the models were too similar to each other to gain much from combining.

Attempt 3 — Wider hyperparameter search + true stacking

We ran a larger randomized search (40 candidate settings x 3-fold cross-validation = 120 model fits) and used a proper 'stacking' model (a Ridge regression that learns how to best combine the three base models). Result: $R^2=0.637$ — actually worse than our simple baseline. **Why:** RandomizedSearchCV picked settings that looked good during cross-validation folds but didn't generalize as well to the actual test set — a classic pitfall on smaller, noisy datasets. Lesson: more complexity is not automatically better.

Attempt 4 — Hourly aggregation (first try) — caught a data leak

We tried predicting *total hourly* energy usage instead of noisy 10-minute readings (averaging out sensor jitter). First result: $R^2=0.987$ — suspiciously perfect. **Investigation:** our lag features (built on 10-minute shifts) ended up almost identical to the current hour's target after aggregation — the model was essentially being handed the answer. This is **data leakage**. We caught it before treating it as a real result — an important part of doing ML responsibly.

Attempt 5 — Hourly aggregation, corrected

We rebuilt the lag features properly, using only genuinely past *hours* (not leaking 10-min data across the aggregation boundary). Corrected result: $R^2=0.412$ — much lower than the leaked version, confirming the earlier 0.987 was indeed fake. This told us hourly aggregation, done honestly, doesn't help for this dataset — the original 10-minute resolution with lag features was actually the better approach.

Attempt 6 — Matching the base paper's outlier-removal approach

Reviewing the base research paper (Abd'Azeez & Olatomiwa, 2025) revealed their key technique: they didn't just cap outliers, they **fully removed** extreme appliance-usage rows using the IQR (Interquartile Range) method, shrinking the target's range dramatically before training. Combined with our own lag/rolling features (which their paper did not use), this is our strongest attempt yet. *[Update this section*

with the final R^2 once run — see the results table below.]

6. Results Summary Table

Attempt	Approach	R2 Score
Baseline	Linear Regression	0.615
Baseline	Random Forest (default)	0.637
Baseline	XGBoost (default)	0.653
Tuned	XGBoost + RandomizedSearchCV	0.656 (best simple model)
Attempt 1	3-model flat average ensemble	0.631
Attempt 2	Outlier capping + weighted ensemble	0.663
Attempt 3	Wider search + true stacking	0.637
Attempt 4	Hourly aggregation (leaked)	0.987 (INVALID — leakage)
Attempt 5	Hourly aggregation (fixed)	0.412
Attempt 6	IQR outlier removal + our lag features (ExtraTrees)	TBD — fill in after running

Current best valid result: $R^2 = 0.656$ (tuned XGBoost, 10-minute resolution, lag/rolling features). Update this line once Attempt 6 is run.

7. Key Talking Points for Team / Review

- This dataset (Candanedo et al., UCI/Kaggle) is genuinely hard — appliance usage depends heavily on human behaviour (someone switching on a kettle), which sensors can't fully capture.
- Our base research paper achieved $R^2=0.7441$ primarily through aggressive IQR-based outlier removal on the target variable, not through a fundamentally different model.
- We independently discovered the value of lag/rolling features (recent past usage), which the base paper did not use at all.
- We correctly identified and rejected a data-leakage result (0.987) instead of reporting it — this shows methodological rigor.
- We learned that ensembling/stacking does not automatically help — it only helps when the combined models make genuinely different kinds of mistakes.

Prepared as an internal walkthrough document. Update Section 5/6 with the final IQR-removal R^2 result before presenting to reviewers.